

Trabajo Práctico Especial - RP-Lang

Instituto Tecnológico de Buenos Aires - Autómatas, Teoría de Lenguajes y
Compiladores (72.39)

Grupo RP-Lang

Ignacio Searles
isearles@itba.edu.ar
64.536

Augusto Barthelemy Solá
abarthelemysola@itba.edu.ar
64.502

Santiago Bassi
sabassi@itba.edu.ar
64.643

21 de junio de 2025

Resumen

El presente informe trata sobre el diseño e implementación de un lenguaje que permite la definición y evaluación de funciones recursivas primitivas. El compilador de **RP-Lang** traduce programas escritos en este lenguaje a código C, aprovechando la expresividad y eficiencia de dicho lenguaje de bajo nivel para ejecutar las funciones definidas.

Contents

1	Introducción	1
2	Modelo computacional	2
2.1	Dominio	2
2.2	Lenguaje	2
3	Implementación	3
3.1	Frontend	3
3.2	Backend	3
3.3	Dificultades encontradas	3
4	Futuras Extensiones	3
5	Conclusiones	3
6	Referencias	3
7	Bibliografía	3

1 Introducción

Las funciones recursivas primitivas constituyen una clase fundamental de funciones total computables, ampliamente estudiadas en teoría de la computación. En este trabajo se presenta **RP-Lang**, un lenguaje de programación diseñado para expresar y evaluar exclusivamente este tipo de funciones.

El desarrollo del lenguaje incluye el diseño de su sintaxis y semántica, así como la implementación de un compilador que traduce programas **RP-Lang** a código en lenguaje C. Para ello, se utilizaron las herramientas Flex y Bison, facilitando la construcción de los analizadores léxico y sintáctico respectivamente.

El objetivo principal del proyecto es proporcionar una herramienta que permita experimentar con funciones recursivas primitivas de forma práctica.

2 Modelo computacional

2.1 Dominio

Desarrollar un lenguaje que permita definir funciones recursivas primitivas (RP). Las funciones RP son de la forma $f : \mathbb{N}^k \rightarrow \mathbb{N}$ (Zahnd, 2022). Las mismas cumplen que están definidas en base a los esquemas recursivos de tipo I y tipo II y composición a partir de las funciones iniciales:

- $suc : \mathbb{N} \rightarrow \mathbb{N}, suc(x) = x + 1$
- $const_k : \mathbb{N} \rightarrow \mathbb{N}, const_k(x) = k$
- $\pi_j^k : \mathbb{N}^k \rightarrow \mathbb{N}, \pi_j^k(x_1, \dots, x_k) = x_j$

Las definiciones de funciones RP pueden ser composiciones o esquemas recursivos:

- Para la composición, si h y g son RP $\implies f(x) = g(h(x))$ es RP.
- Para el esquema recursivo de tipo I, si $g : \mathbb{N}^2 \rightarrow \mathbb{N}$ es RP $\implies f : \mathbb{N} \rightarrow \mathbb{N}$ tal que $f(0) = k \in \mathbb{N}$ y $f(n+1) = g(n, f(n))$ es RP.
- Para el esquema recursivo de tipo II, $g : \mathbb{N}^{n+2} \rightarrow \mathbb{N}$ y $q : \mathbb{N}^n \rightarrow \mathbb{N}$ son RP $\implies f : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ tal que $f(x_1, \dots, x_n, 0) = q(x_1, \dots, x_n)$ y $f(x_1, \dots, x_n, y+1) = g(x_1, \dots, x_n, y, f(x_1, \dots, x_n, y))$ es RP.

El lenguaje permite evaluar cualquier función RP que se haya definido. Haciendo uso de las definiciones de las mismas para evaluarlas recursivamente y obtener un valor de salida.

2.2 Lenguaje

Un programa de **RP-Lang** (extensión recomendada **.rp**), está compuesto por definiciones, evaluaciones e **#include**. Los mismos puede aparecer intercalados en cualquier orden, mientras se cumpla que no se utilicen funciones no definidas en una definición. A continuación se listan construcciones del lenguaje:

- Se pueden definir funciones partiendo de las funciones básicas.
- Se pueden definir funciones mediante composición de funciones.
- Se pueden definir funciones mediante esquemas recursivos de tipo I y II.
- Se pueden evaluar funciones utilizando constantes o entradas por teclado.
- Se pueden importar funciones desde otros archivos.
- Las funciones binarias se pueden evaluar mediante la syntaxis: **(expr1 func expr2)** donde $f : \mathbb{N}^2 \rightarrow \mathbb{N}$.
- Se pueden incluir comentarios en el código.

Las funciones pueden definirse mediante expresiones compuestas o mediante esquemas recursivos. A continuación se muestran ejemplos representativos de cada caso.

Composición

```
#include times.rp

def pow3(x):
=> pow3(x) = (x * (x * x))

//this will prompt the user for input
pow3(input)
```

Recursión (esquema tipo I)

```
#include times.rp

def !(n):
-> !(0) = 1
-> !(n + 1) = *(suc(n), !(n))

!(4)
```

Recursión (esquema tipo II)

```
#include times.rp

def ^(x, y):
-> ^(x, 0) = 1
-> ^(x, y + 1) = *(x, ^(x, y))

(2 ^ 5)
```

3 Implementación

3.1 Frontend

3.2 Backend

3.3 Dificultades encontradas

4 Futuras Extensiones

5 Conclusiones

6 Referencias

7 Bibliografía

A continuación, se enumeran las fuentes consultadas para la elaboración de este trabajo.

- Zahnd, M. (2022). GitHub - mzahnd/apuntes-9335-logica-computacional: Apuntes de la materia Lógica Computacional (93.35) del ITBA. GitHub.